

1] Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as ps, top, and /proc. Document the observations.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;

    printf("Parent process started.\n");
    printf("Parent PID : %d\n", getpid());
    printf("Parent PPID : %d\n", getppid());

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        // Child process
        printf("\n--- CHILD PROCESS ---\n");
        printf("Child PID : %d\n", getpid());
        printf("Child PPID : %d\n", getppid());

        printf("Child is now running.\n");
        printf("Child will sleep for 10 seconds (waiting state).\n");

        sleep(10);

        printf("Child is running again after sleep.\n");

        printf("Child is terminating.\n");

        return 0;
    }
    else
    {
        // Parent process
        printf("\n--- PARENT PROCESS ---\n");
        printf("Parent PID : %d\n", getpid());
        printf("Child PID : %d\n", pid);

        printf("Parent is waiting for child to finish.\n");

        wait(NULL);

        printf("Child has terminated.\n");
        printf("Parent is terminating.\n");
    }
}
```

```
monish_putti@monhh: ~  
monish_putti@monhh:~$ nano process_states.c  
monish_putti@monhh:~$ gcc process_states.c -o process_states  
monish_putti@monhh:~$ ./process_states  
Parent process started.  
Parent PID : 553  
Parent PPID : 362  
  
--- PARENT PROCESS ---  
Parent PID : 553  
Child PID : 554  
Parent is waiting for child to finish.  
  
--- CHILD PROCESS ---  
Child PID : 554  
Child PPID : 553  
Child is now running.  
Child will sleep for 10 seconds (waiting state).  
Child is running again after sleep.  
Child is terminating.  
Child has terminated.  
Parent is terminating.  
monish_putti@monhh:~$
```

```
monish_putti@monhh: ~  
monish_putti@monhh:~$ nano process_states.c  
monish_putti@monhh:~$ gcc process_states.c -o process_states  
monish_putti@monhh:~$ ./process_states &  
[1] 719  
Parent process started.  
Parent PID : 719  
Parent PPID : 362  
  
--- PARENT PROCESS ---  
Parent PID : 719  
Child PID : 721  
Parent is waiting for child to finish.  
  
--- CHILD PROCESS ---  
Child PID : 721  
Child PPID : 719  
Child is now running.  
Child will sleep for 10 seconds (waiting state).  
monish_putti@monhh:~$ ps -o pid,ppid,state,cmd -C process_states  
PID PPID S CMD  
719 362 S ./process_states  
721 719 S ./process_states  
monish_putti@monhh:~$ grep -E "Name|State|Pid|PPid" /proc/701/status  
grep: /proc/701/status: No such file or directory  
monish_putti@monhh:~$
```

2] Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.

Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int i;

    printf("Parent PID: %d\n", getpid());

    for (i = 0; i < 3; i++)
    {
        pid_t pid = fork();

        if (pid < 0)
        {
            perror("fork failed");
            exit(1);
        }

        if (pid == 0)
        {
            printf("Child %d created: PID = %d, PPID = %d\n",
                i + 1, getpid(), getppid());

            sleep(i + 1);

            printf("Child %d terminating: PID = %d\n",
                i + 1, getpid());

            exit(10 + i);
        }
    }

    printf("\nParent is waiting for children using wait().\n");

    for (i = 0; i < 3; i++)
    {
        int status;
        pid_t child_pid;

        child_pid = wait(&status);

        printf("Parent collected child PID = %d\n", child_pid);

        if (WIFEXITED(status))
        {
            printf("Exit status = %d\n", WEXITSTATUS(status));
        }
    }

    printf("All children have been collected.\n");
    printf("Parent terminating.\n");

    return 0;
}

```

```

monish_putti@monhh: ~/OSSP_Practical
monish_putti@monhh:~$ cd ~/OSSP_Practical
-bash: cd: /home/monish_putti/OSSP_Practical: No such file or directory
monish_putti@monhh:~$ mkdir -p ~/OSSP_Practical
monish_putti@monhh:~$ cd ~/OSSP_Practical
monish_putti@monhh:~/OSSP_Practical$ ls
monish_putti@monhh:~/OSSP_Practical$ nano multiple_wait.c
monish_putti@monhh:~/OSSP_Practical$ gcc multiple_wait.c -o multiple_wait
monish_putti@monhh:~/OSSP_Practical$ ./multiple_wait
Parent PID: 867
Child 1 created: PID = 868, PPID = 867
Child 2 created: PID = 869, PPID = 867
Child 3 created: PID = 870, PPID = 867

Parent is waiting for children using wait().
Child 1 terminating: PID = 868
Parent collected child PID = 868
Exit status = 10
Child 2 terminating: PID = 869
Parent collected child PID = 869
Exit status = 11
Child 3 terminating: PID = 870
Parent collected child PID = 870
Exit status = 12
All children have been collected.
Parent terminating.
monish_putti@monhh:~/OSSP_Practical$

```

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t children[3];
    int i;

    printf("Parent PID: %d\n", getpid());

    for (i = 0; i < 3; i++)
    {
        children[i] = fork();

        if (children[i] < 0)
        {
            perror("fork failed");
            exit(1);
        }

        if (children[i] == 0)
        {
            printf("Child %d created: PID = %d, PPID = %d\n",
                i + 1, getpid(), getppid());

            sleep(3 - i);

            printf("Child %d terminating: PID = %d\n",
                i + 1, getpid());

            exit(20 + i);
        }
    }

    printf("\nParent is waiting for specific children using waitpid().\n");

    for (i = 0; i < 3; i++)
    {
        int status;

        waitpid(children[i], &status, 0);

        printf("Parent collected specific child PID = %d\n",
            children[i]);

        if (WIFEXITED(status))
        {
            printf("Exit status = %d\n",
                WEXITSTATUS(status));
        }
    }

    printf("All children have been collected.\n");
    printf("Parent terminating.\n");

    return 0;
}

```

```
monish_putti@monhh: ~/OSSP_Practical
Parent collected child PID = 868
Exit status = 10
Child 2 terminating: PID = 869
Parent collected child PID = 869
Exit status = 11
Child 3 terminating: PID = 870
Parent collected child PID = 870
Exit status = 12
All children have been collected.
Parent terminating.
monish_putti@monhh:~/OSSP_Practical$ nano multiple_waitpid.c
monish_putti@monhh:~/OSSP_Practical$ gcc multiple_waitpid.c -o multiple_waitpid
monish_putti@monhh:~/OSSP_Practical$ ./multiple_waitpid
Parent PID: 914
Child 1 created: PID = 915, PPID = 914
Child 2 created: PID = 916, PPID = 914

Parent is waiting for specific children using waitpid().
Child 3 created: PID = 917, PPID = 914
Child 3 terminating: PID = 917
Child 2 terminating: PID = 916
Child 1 terminating: PID = 915
Parent collected specific child PID = 915
Exit status = 20
Parent collected specific child PID = 916
Exit status = 21
Parent collected specific child PID = 917
Exit status = 22
All children have been collected.
Parent terminating.
monish_putti@monhh:~/OSSP_Practical$
```

```
monish_putti@monhh: ~/OSSP_Practical
Parent collected child PID = 868
Exit status = 10
Child 2 terminating: PID = 869
Parent collected child PID = 869
Exit status = 11
Child 3 terminating: PID = 870
Parent collected child PID = 870
Exit status = 12
All children have been collected.
Parent terminating.
monish_putti@monhh:~/OSSP_Practical$ nano multiple_waitpid.c
monish_putti@monhh:~/OSSP_Practical$ gcc multiple_waitpid.c -o multiple_waitpid
monish_putti@monhh:~/OSSP_Practical$ ./multiple_waitpid
Parent PID: 914
Child 1 created: PID = 915, PPID = 914
Child 2 created: PID = 916, PPID = 914

Parent is waiting for specific children using waitpid().
Child 3 created: PID = 917, PPID = 914
Child 3 terminating: PID = 917
Child 2 terminating: PID = 916
Child 1 terminating: PID = 915
Parent collected specific child PID = 915
Exit status = 20
Parent collected specific child PID = 916
Exit status = 21
Parent collected specific child PID = 917
Exit status = 22
All children have been collected.
Parent terminating.
monish_putti@monhh:~/OSSP_Practical$
```

1. Create the zombie program

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main()
{
    pid_t pid;

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        printf("\n--- CHILD PROCESS ---\n");
        printf("Child PID : %d\n", getpid());
        printf("Child PPID : %d\n", getppid());

        printf("Child is terminating.\n");

        exit(0);
    }
    else
    {
        printf("\n--- PARENT PROCESS ---\n");
        printf("Parent PID : %d\n", getpid());
        printf("Child PID : %d\n", pid);

        printf("Parent is NOT calling wait().\n");
        printf("Child will become a zombie.\n");

        sleep(30);

        printf("Parent terminating.\n");
    }

    return 0;
}
```

```
monish_putti@monhh: ~/OSSP_Practical
Parent terminating.
monish_putti@monhh:~/OSSP_Practical$ nano zombie.c
monish_putti@monhh:~/OSSP_Practical$ gcc zombie.c -o zombie
monish_putti@monhh:~/OSSP_Practical$ ./zombie &
[1] 968

--- PARENT PROCESS ---
Parent PID : 968
Child PID : 970
Parent is NOT calling wait().
Child will become a zombie.
--- CHILD PROCESS ---
Child PID : 970
Child PPID : 968
Child is terminating.
monish_putti@monhh:~/OSSP_Practical$ ps -o pid,ppid,state,stat,cmd -C zombie
  PID   PPID  S  STAT  CMD
  968    766  S   S     ./zombie
  970    968  Z   Z     [zombie] <defunct>
monish_putti@monhh:~/OSSP_Practical$ grep -E "Name|State|Pid|PPid" /proc/951/status
grep: /proc/951/status: No such file or directory
monish_putti@monhh:~/OSSP_Practical$ Parent terminating.
```

2. Actually eliminate the zombie

```
monish_putti@monhh: ~/OSSP_Practical
GNU nano 8.7.1 zombie_fixed.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        printf("\n--- CHILD PROCESS ---\n");
        printf("Child PID : %d\n", getpid());
        printf("Child PPID : %d\n", getppid());

        printf("Child is terminating.\n");

        exit(0);
    }
    else
    {
        printf("\n--- PARENT PROCESS ---\n");
        printf("Parent PID : %d\n", getpid());
        printf("Child PID : %d\n", pid);

        printf("Parent is waiting for child using wait().\n");

        wait(NULL);

        printf("Child has been collected.\n");
        printf("Zombie process eliminated.\n");

        printf("Parent terminating.\n");
    }

    return 0;
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line M-E Redo

monish_putti@monhh: ~/OSSP_Practical
monish_putti@monhh:~/OSSP_Practical$ nano zombie_fixed.c
monish_putti@monhh:~/OSSP_Practical$ gcc zombie_fixed.c -o zombie_fixed
monish_putti@monhh:~/OSSP_Practical$ ./zombie_fixed

--- PARENT PROCESS ---
Parent PID : 1092
Child PID : 1093
Parent is waiting for child using wait().

--- CHILD PROCESS ---
Child PID : 1093
Child PPID : 1092
Child is terminating.
Child has been collected.
Zombie process eliminated.
Parent terminating.
monish_putti@monhh:~/OSSP_Practical$
```